Architectural & Technical System

Technical Architecture

Architecture Overview

IntegrateWise — Executive Architecture

Canonical statement: IntegrateWise is a **Knowledge Workspace over the Spine, empowered by AI and governed by approvals**.

---

1. What the system is

IntegrateWise is designed to make business truth, context, and action operate in one governed loop.

It combines:

- **L0 Onboarding** — establishes tenant identity, department, industry, and schema direction
- **L1 Workspace** — the daily operating surface where users work
- **L2 Cognitive** — the intelligence layer that reasons, proposes, and waits for approval
- **L3 Platform** — the controlled backend path where all data is ingested, normalized, stored, and re-ingested

The architecture works because the **Workspace is the product surface**, the **Spine is the source of truth**, and **AI only acts through governed approval paths**.

---

2. Core architectural rule

> **All durable business value in IntegrateWise must be created through a Spine-based flow.**

That means no connector, UI module, AI session, or workflow may create business truth outside the canonical backend path.

---

3. Runtime model

Landing → Auth → L0 Onboarding → L1 Workspace → L2 Cognitive → L3 Platform

Layer roles

- **L0 Onboarding**
  Resolves tenant context, department base schema, industry overrides, and connector relevance.
- **L1 Workspace**
  Renders dashboards, modules, approvals, signals, and entity views from Spine-backed runtime projections.
- **L2 Cognitive**
  Builds Entity 360, generates signals and proposals, checks policy, routes approvals, and coordinates action.
- **L3 Platform**
  Runs gateway, connectors, loader, queueing, 8-stage pipeline, Spine, knowledge services, workflow/BFF, and re-ingestion.

---

4. Data architecture in one line

Ingest → Normalize → Store → Render → Think → Govern / HITL → Act → Re-ingest

What this means

- external data enters through connectors, uploads, webhooks, or AI capture
- all meaningful data passes through the mandatory 8-stage pipeline
- canonical truth is written to the Spine

- workspace and cognitive layers read from Spine-backed projections
- AI reasons over Entity 360 and linked evidence
- actions are executed only after approval when policy requires it
- outcomes return through re-ingestion so truth stays current

---

5. The three flows

Flow A — Structured operational truth

Used for CRM, finance, support, project, and other structured systems.

Path: Connector → Pipeline → Spine → Workspace / Entity 360 / Signals

Flow B — Unstructured context and evidence

Used for documents, emails, chats, files, and transcripts.

Path: Ingest → Extraction / chunking / linking → Knowledge + Spine references → Entity 360

Flow C — AI / MCP / knowledge-first flow

Used for AI sessions, MCP sessions, and governed memory.

Path: Capture → Triage → Approved knowledge / decision support → Approved action → Re-ingestion → Spine

Critical rule: Flow C never writes directly to Spine truth.

---

6. The Spine

The Spine is the canonical truth boundary.

It stores:

- entities
- relationships
- schema observations
- structured operational truth
- linked references required for runtime views

If truth is not written to the Spine through the controlled pipeline, it is not part of the official platform state.

---

7. Workspace loading rule

> **The Workspace Layer never loads from connectors directly.**

L1 becomes usable only after **Creamy hydration** has passed through:

connector auth → loader → queue → pipeline → normalizer → Spine write

The workspace then reads:

- tenant config
- readiness
- dashboard projections
- module projections

- signals
- Entity 360-backed views

This is why the workspace remains stable, fast, and governed by normalized truth.

---

8. Department and industry activation

The architecture is schema-activated at onboarding.

- **Department** chooses the base runtime schema family
- **Industry** modifies that family through additive overrides

This determines:

- allowed entity types
- priority fields
- workspace emphasis
- organic capabilities exposed to users
- what signals and reasoning patterns become meaningful

---

9. Cognitive loop

The intelligence loop is:

Entity 360 → Think → Govern → HITL → Act → Adjust

Meaning

- **Think** generates grounded proposals
- **Govern** applies policy and approval rules
- **HITL** captures human approval where required
- **Act** executes approved changes externally
- **Adjust** learns from outcomes and decisions

The result is then re-ingested back through the canonical data path.

---

10. Executive architecture summary

IntegrateWise is a **workspace-first, Spine-based, approval-governed system**.

Its architecture ensures that:

- onboarding determines the correct runtime schema
- connectors supply data but never define truth
- the pipeline normalizes all meaningful input
- the Spine is the single source of truth
- the workspace renders normalized business reality
- the cognitive layer reasons over Entity 360, not raw payloads
- AI never bypasses governance
- action outcomes always return through re-ingestion

Final statement

IntegrateWise architecture is designed so that truth is created only through the Spine, work happens in the Workspace, intelligence operates in the Cognitive layer, and every approved action returns through the platform so the system stays consistent over time.

OnBoarding Phase

Entry & Ingress

MCP Connector

Daily Data Sync Loop

Integration Connectors

Creamy / Initial Data Loading

Domain Connectors

Accelarators

Custom Connectors

WebHooks

3 Flows

Structured Data - Flow A

UnStructured Data - Flow B

AI Conversations & Sessions - Flow C

WorkSpace Documentation

Industry and Department,

Daily WorkSpace - Layer 1

Intelligent Layer

Spine Architecture

Intelligent Schema Generator

The Cognitive Family

AI Loader

Normalizer

N8N Pipeline or 8 Stage PipeLine

Triage Center

No Hallucination Library & Knowledge UI

Entity 360

AI Insights

Think

Cognitive Twin

Human in the Loop

Act - Govern - Adjust - Configure

Evidence Layer

Slack Centered Application

Signals connecting to Flow A & B

Services

Tenant and Admin Managed Services

Billing and Paywall Activations

Repeat - Loop

Tools - Workspace and Workspace to Tools

IntegrateWise — Data Flow KT v1.1

IntegrateWise — Data Flow KT v1.1

Status: Canonical internal reference
Purpose: Define how data should flow through IntegrateWise from onboarding to workspace, Spine, cognitive loop, approval, action, and re-ingestion.

---

1. Purpose

This KT note defines the correct end-to-end data flow for IntegrateWise so product, engineering, connector, data, and AI teams all work from the same model.

The governing rule is:

All durable business value in IntegrateWise must be created through a Spine-based flow.

The canonical runtime chain is:

Landing → Auth → L0 Onboarding → L1 Workspace → L2 Cognitive → L3 Platform

The canonical system loop is:

LOAD → NORMALIZE → STORE → THINK → REVIEW & APPROVE → ACT → REPEAT

---

2. Canonical Product Statement

IntegrateWise is a **Knowledge Workspace over the Spine, empowered by AI and governed by approvals**.

That means:

- **Workspace** is where users operate
- **Spine** is the structured source of truth
- **Knowledge** carries linked unstructured context and approved compounding context
- **AI / Cognitive** reasons over canonical data
- **Approvals** gate execution
- **Actions** must reconcile back through the platform so the Spine stays current

---

3. Core Principles

3.1 The Spine is the single source of truth

The official architecture is explicit that the Spine is the foundation and that ingestion, context, intelligence, backend services, and AI all derive value only when they operate on Spine-shaped data.

3.2 No data may bypass Layer 3

Every piece of information must pass through the controlled L3 pipeline before it can appear in L2 or L1. There are no shortcuts or side write paths.

3.3 Flow C never writes directly to the Spine

Flow A and Flow B can create truth through pipeline-normalized writes. Flow C contributes knowledge and decision context; truth updates from Flow C only happen through approved actions that go back through Act → pipeline re-ingestion → Spine.

3.4 The workspace is not a truth layer

L1 is where people work, but it is dynamically generated from Spine data, Depth Matrix, BFF projections, and linked knowledge. L1 does not load raw connector data directly and does not maintain a separate SSOT.

3.5 No action executes without approval

The cognitive loop is always Think → Govern → HITL → Act / Adjust. No durable external mutation is valid without the approval path when policy requires it.

---

4. Official Runtime Architecture

The official runtime layer order is:

Experience → Gateway → Workspace Runtime → External Connectivity → Data Plane → SSOT / Spine → Cognitive → AI Providers

Layer roles

- **L0 Onboarding**: collects context, seeds `tenant_spine_config`, and prepares workspace + schema direction
- **L1 Workspace**: personalized work layer with modules, dashboards, approvals, and entity views driven by Spine + Depth Matrix
- **L2 Cognitive**: Think, Govern, HITL, Act, Adjust operating on Spine + linked context
- **L3 Backend**: Gateway, connectors, loader, pipeline, Spine, knowledge, and background services; the only entry point for external data into the system

---

5. Canonical End-to-End Loop

The platform loop is:

Load → Normalize → Store → Think → Govern → HITL → Act → Adjust → Repeat

Expanded meaning

1. **Load**
   Data enters through connectors, files, webhooks, MCP sessions, or AI session capture.
2. **Normalize**
   The pipeline transforms raw source payloads into tenant-valid, schema-valid, entity-valid structures.
3. **Store**
   Canonical truth is written to Spine; linked unstructured context goes to Knowledge.
4. **Think**
   Entity 360, signals, and approved context are used to generate proposals and insights.
5. **Govern / HITL**
   Policies are checked and humans approve or reject actions.
6. **Act**
   Approved actions execute through controlled integration channels.
7. **Adjust / Repeat**
   Outcomes are recorded, decision memory improves the next cycle, and re-ingestion reconciles truth.

---

6. End-to-End Flow from Landing to Cognitive

The full system flow is:

1. User lands on marketing or app entry.
2. Auth resolves session and onboarding status.
3. L0 onboarding seeds `tenant_spine_config`.
4. User selects and authenticates connectors.
5. The first `phase: "creamy"` hydration starts.
6. Loader enqueues records to `PIPELINE_QUEUE`.
7. Normalizer runs all 8 stages.
8. `spine-v2` writes canonical truth and schema observations.
9. The initial workspace becomes usable from Spine-backed creamy data.
10. Needed and Delta continue in the background.
11. Entity 360 is read from Spine + Knowledge + Signals.
12. L2 cognitive logic activates after creamy and connector threshold rules are met.

---

7. Onboarding and Schema Seeding (L0)

Onboarding is the point where tenant reality is declared.

It determines:

- who the tenant is
- industry
- department / domains
- desired workspace
- connector relevance
- whether knowledge UI should link into Entity 360
- the initial Depth Matrix / Spine direction

The system stores this in tenant-scoped configuration, especially `tenant_spine_config`, which later drives workspace navigation, readiness, entity types, and allowed schema behavior.

Onboarding output

Onboarding does not only collect profile information. It actively seeds the future runtime by creating the initial workspace and tenant schema basis. The fixed runtime path explicitly states that Landing / Auth / Onboarding create the Initial Workspace and seed `tenant_spine_config`.

---

8. Connector Selection, Auth, and Sync Kickoff

The connect stage must work like this:

1. Connectors are shown based on **industry + department**
2. User authenticates a connector
3. Installation is bound to the tenant
4. Connected connector state is persisted
5. The first `phase: "creamy"` sync is triggered immediately on success

The internal landing-flow doc explicitly describes the chain:

- connectors filtered by industry + department
- authentication callback
- resource types + traits determine what data is allowed
- creamy runs through loader → queue → 8-stage pipeline → normalizer → Spine
- workspace then reads BFF dashboard, readiness, tenant-config, modules, and Entity 360 sources

---

9. The Three Data Flows

9.1 Flow A — Structured data

Flow A is structured operational data from external systems such as CRM, billing, support, productivity, and related business tools. It lands as truth in the Spine.

Flow A path

Connection success → connector-sync → connector / loader → PIPELINE_QUEUE → Normalizer 8 stages → Spine truth → Initial Workspace → Needed / Delta → Entity 360 → Think / Twin

Flow A role

Flow A is the main truth creation path for operational records. It is the path that powers dashboard lists, entity records, workspace views, and most signals.

9.2 Flow B — Unstructured content

Flow B handles documents, emails, files, and other unstructured context. It goes through ingestion, linking, and Knowledge storage while also creating Spine references that can be used in Entity 360. The full landing-flow diagram describes Flow B as: **Ingest → entity linking → Spine + Knowledge → Entity 360**.

Flow B rule

Flow B enriches context and evidence. It does not replace structured truth.

9.3 Flow C — AI sessions, MCP, and compounding knowledge

Flow C handles MCP / AI / chat / triage / compounding knowledge. It follows the knowledge-first path and never writes directly to Spine truth. The landing-flow architecture shows Flow C as: **MCP/AI → D1 → Triage Center Bot → triage space → approve → compounding**.

Flow C rule

Only approved actions from Flow C can influence truth, and they must do so through Act → connector write-back → pipeline re-ingestion → Spine.

---

10. Workspace Layer Data Loading (L1)

This is the missing section now added to the KT.

The workspace does **not** load raw connector data directly. L1 is a **projection layer** that becomes usable only after normalized, Spine-backed data exists. The docs are explicit: all information from external sources must pass through L3 and be written to Spine or Knowledge before it can appear in L2 or L1.

10.1 The workspace loading principle

L1 is the user-facing work surface, but it is not:

- the ingestion layer
- the truth layer
- a direct connector read layer

Its job is to render context-aware views of already normalized data coming from:

- Spine truth
- BFF projections
- linked Knowledge
- Signals
- optional compounding space when linked into Entity 360

10.2 What makes the workspace “ready”

The workspace becomes usable after the initial **Creamy** phase has passed through:

connector auth → loader → queue → 8-stage pipeline → normalizer → Spine write

The fixed runtime path explicitly says:

- initial workspace is created from onboarding
- real connector auth succeeds
- `phase: "creamy"` hydration is triggered
- canonical truth is written
- the **Initial Workspace becomes usable after Creamy**
- Needed and Delta continue later in the background

10.3 The exact L1 loading chain

The correct loading sequence is:

Landing / Auth / Onboarding → Connector selection → Authentication → Creamy hydration → Spine write → BFF / tenant-config / readiness / dashboard → Workspace shell + modules → Entity 360 + signals → L2 activation

10.4 The four loading surfaces inside L1

A. Workspace shell load

This is the first visible workspace load.

It includes:

- workspace shell
- sidebar / nav
- tenant context
- active domain / view
- work vs personal mode
- readiness state

The landing-flow doc states that L1 is built from **WorkspaceShell**, nav from `tenant_spine_config`, BFF dashboard/readiness/HITL/analytics, and ContentRouter to domain modules.

B. Home dashboard load

This is the next visible layer.

It includes:

- summary cards
- approvals count
- intelligence count
- focus items
- dashboard widgets
- readiness / dashboard state

The BFF surface explicitly serves:

- `GET /api/v1/workspace/tenant-config`
- `GET /api/v1/workspace/dashboard`
- `GET /api/v1/workspace/readiness`
- HITL queue and actions

C. Module data load

This is where module views load.

Examples seen in current workspace design and docs:

- Accounts / Projects
- Meetings
- Docs
- Tasks
- Calendar
- Notes
- Knowledge Space
- Team
- Pipeline
- Risks
- Expansion
- Intelligence surfaces

These are domain projections rendered from Spine-backed runtime data, not live connector API reads. The full flow docs describe the workspace as dashboard, list views, modules, and Entity 360 built from Spine and internal stores.

D. Intelligence overlays back into L1

L1 also shows intelligence before the user opens L2, including:

- banners

- signal indicators
- approvals
- feed summaries
- attention queues

The phased flow doc says dashboard and list views are built from Spine and BFF, while proposals from Think appear in the workspace and the user approves, edits, or rejects there.

10.5 Official L1 data sources

The workspace reads from these internal sources:

1. **Spine** — domain tables, core truth, projections
2. **Knowledge** — Flow B entity-linked chunks
3. **Knowledge UI / compounding space** — only when `link_knowledge_ui_to_entity360` is true
4. **Signals** — derived from Normalizer + Think

10.6 Workspace loading happens in phases

L1 loading is phased.

Phase 1 — Shell ready

Auth, tenant, onboarding completion, nav, and workspace shell resolve.

Phase 2 — Creamy workspace ready

First useful normalized data lands; user can work.

Phase 3 — Module hydration

Dashboard and module views begin filling from BFF + Spine projections.

Phase 4 — Needed expansion

The next layer of business truth expands entity and historical coverage.

Phase 5 — Delta freshness

Incremental refresh keeps L1 current.

10.7 What should load immediately vs later

Load immediately after Creamy

- shell and nav
- tenant identity
- dashboard cards
- initial approvals and intelligence feed
- first account/task/meeting/contact projections
- first entity-ready work surfaces

Load later through Needed / Delta

- deeper history
- wider entity coverage
- more complete module views
- richer Entity 360
- stronger signal quality
- better cognitive recommendations

10.8 L1 / L2 activation dependency

The official rule is:

- **L1 can render once Creamy truth exists**
- **L2 intelligence should activate only after Creamy is complete and at least 2–3 meaningful connectors are loaded**

This is the dependency that prevents empty or weak intelligence on shallow data.

10.9 User-visible interpretation

The user-visible meaning of workspace loading is:

1. Onboarding decides the lens
2. Connector auth makes the data real
3. Creamy gives first value fast
4. Spine becomes the source
5. Workspace renders that truth
6. Needed and Delta deepen the workspace silently
7. L2 activates once enough truth exists

10.10 One-line workspace rule

The Workspace Layer does not load from connectors directly; it loads from Spine-backed runtime projections after Creamy hydration has passed through connector auth, loader, pipeline, normalizer, and canonical Spine write.

---

11. The Mandatory 8-Stage Pipeline

Every meaningful incoming record must pass through the 8-stage pipeline:

1. Analyze
2. Classify
3. Filter
4. Refine
5. Extract
6. Validate
7. Sanity
8. Sectorize

This is the mandatory normalization path for all canonical truth creation. Raw external events do not bypass it.

---

12. Entity 360

Entity 360 is a read-time fusion view.

It is built from:

- Spine
- Knowledge
- Signals
- optional compounding space when `link_knowledge_ui_to_entity360` is true

Rule

Entity 360 is a **read model**, not a materialized truth table and not a write surface. The docs explicitly describe it as built at read time.

---

13. Cognitive Loop (L2)

The cognitive loop is:

Entity 360 → Think → Govern → HITL → Act / Adjust → Spine updated

Think

Consumes Entity 360 and linked knowledge to generate proposals and evidence refs.

Govern

Performs policy checks and ensures approval token gating.

HITL

User approves, edits, or rejects proposals in the workspace.

Act / Adjust

Approve leads to execution; deny leads to feedback / decision memory. The docs describe loop closure as Act updating the system and Adjust feeding future thinking.

---

14. Approval and Action Model

No action is complete until the result has been reconciled through the platform.

Correct action path

1. Proposal created
2. Govern checks policy
3. User approves in workspace
4. Act executes external mutation
5. Result re-enters pipeline
6. Spine updates
7. Workspace refreshes from canonical truth

This prevents internal truth from drifting away from actual external state.

---

15. Storage Responsibility Model

Spine

Canonical structured truth, entity tables, relationships, schema observations.

Knowledge

Unstructured chunks, evidence, entity-linked contextual data.

Knowledge UI / compounding space

Approved Flow C knowledge and memory layer, optionally linked into Entity 360.

Audit / actions / decisions

Approvals, denials, action records, and traceability.

L1 rule

The dashboard is powered from internal Spine-backed data via the BFF. It does not make live multi-system calls for each initial load; it reads the normalized internal copy and internal projections.

---

16. Anti-Patterns to Avoid

Do not allow these:

1. **Direct source-to-truth writes**
   Connectors or webhooks must not directly write business truth outside the pipeline.
2. **AI direct-to-Spine writes**
   Flow C must never mutate truth directly.
3. **Shadow schemas**
   No parallel truth models in modules, connectors, or AI services. The Spine is the foundation.
4. **Workspace reading external systems directly**
   L1 must read from BFF / Spine-backed projections, not from raw connector APIs.
5. **Entity 360 as write surface**
   Entity 360 is for fusion and reasoning only.
6. **Approval bypass**
   No governed action may execute without the correct approval path.

---

17. Code / Runtime Map

For engineers tracing implementation, the main runtime map is:

- onboarding seeds tenant config and schema direction
- connector auth binds installation
- sync triggers creamy
- loader enqueues to pipeline
- normalizer runs 8 stages

- Spine writes truth
- BFF exposes tenant-config, readiness, dashboard
- WorkspaceShell + ContentRouter render L1
- Entity 360 reads Spine + Knowledge + Signals
- Think / Govern / HITL / Act run on top of that truth

---

18. Acceptance Checklist

Tenant and onboarding

- tenant config is seeded correctly
- onboarding choices define runtime scope
- workspace shell uses tenant config and readiness

Connector and sync

- connector auth binds to tenant
- connection success triggers creamy
- source data enters the queue path

Pipeline and truth

- all records pass through all 8 stages
- truth writes land in Spine
- Flow B writes linked Knowledge context
- Flow C remains knowledge-first until approval-driven action

Workspace loading

- L1 shell reads BFF / tenant-config / readiness
- dashboard uses Spine-backed projections
- connectors do not directly populate L1
- creamy is the gate for usable workspace
- needed and delta deepen the workspace in background
- signals in L1 come from normalized data and Think
- L2 activation is gated until creamy + connector threshold

Cognitive and action

- Entity 360 is read-time fusion only
- Think reads canonical truth
- Govern and HITL gate action
- Act results re-enter the pipeline
- final workspace state reflects reconciled truth

---

19. One-Sentence KT Summary

IntegrateWise data must always enter through controlled L3 ingest paths, pass through the mandatory normalization pipeline, become canonical truth in the Spine, surface into the workspace through BFF and Spine-backed projections, power Entity 360 and the cognitive loop, route actions through approval, and return through re-ingestion so the Spine remains the only source of truth.

If you want, the next step is turning this into a polished internal artifact with a title page, version block, diagrams, and a short “L1/L2 screenshots mapping” appendix.

Appendix A — Code Map

Agreed. Add this as the appendix.

Appendix A — Code Map

This appendix maps the canonical data flow to the main implementation files so engineers can trace behavior in code.

1. Onboarding and tenant schema resolution

Responsibility	File / Area
L0 onboarding flow and tenant activation	`apps/web/src/components/onboarding/`
Tenant schema resolution	`packages/types/src/adaptive-schema.ts`
Tenant runtime configuration	`tenant_spine_config` usage across workflow / onboarding services

What to trace

- how department + industry resolve the tenant schema
- how onboarding completion unlocks workspace readiness
- how connector setup is tied to tenant context

---

2. Connector auth and sync kickoff

Responsibility	File / Area
Connector auth and sync production	`services/connector/src/sync-producer.ts`
Connector-specific handlers	`services/loader/src/handlers/`

What to trace

- connector installation binding to tenant
- creamy / needed / delta sync behavior
- initial queue dispatch after auth success

---

3. Structured data flow (Flow A)

Responsibility	File / Area

Structured ingestion handlers	`services/loader/src/handlers/hubspot.ts`, `salesforce.ts`, etc.
Canonical field mapping	`services/loader/src/pipeline-stages.ts`
Pipeline orchestration	`services/loader/src/pipeline.ts`
8-stage normalization	`services/normalizer/src/index.ts`
Spine write routing	`services/spine-v2/src/index.ts`

What to trace

- raw webhook / poll payload enters loader
- payload goes through canonical mapping
- `run8StagePipeline()` executes
- normalized entity is routed to the correct Spine table
- resulting truth becomes available to workspace and cognitive layers

---

4. The 8-stage pipeline

Stage	Meaning	Main file
Analyze	inspect raw payload	`services/normalizer/src/index.ts`
Classify	determine entity / flow	`services/normalizer/src/index.ts`
Filter	tenant/schema filtering	`services/normalizer/src/index.ts`
Refine	normalize field shapes	`services/normalizer/src/index.ts`
Extract	pull canonical entity data	`services/normalizer/src/index.ts`
Validate	schema + required field checks	`services/normalizer/src/index.ts`
Sanity	business/data integrity checks	`services/normalizer/src/index.ts`
Sectorize	final destination routing	`services/normalizer/src/index.ts`, `services/spine-v2/src/index.ts`

Developer note
This is the mandatory path for truth creation. No connector should bypass it.

---

5. Spine / SSOT routing

Responsibility	File / Area
Entity type to table routing	`services/spine-v2/src/index.ts`
Canonical truth persistence	`services/spine-v2/src/index.ts`

What to trace

- `ENTITY_TYPE_TO_TABLE`
- destination table resolution
- structured truth writes
- prevention of non-canonical write paths

---

6. Unstructured content flow (Flow B)

Responsibility	File / Area
Chunking	`services/knowledge/src/chunking/`
Embeddings	`services/knowledge/src/embedding/`
Knowledge ingestion	`services/knowledge/src/`
Entity linking / metadata projection	knowledge + Spine-linked context paths

What to trace

- file or content enters ingest
- text extraction / chunking happens
- embeddings are created
- searchable content lands in Knowledge
- linked metadata / references are attached to Spine entities

Rule reminder
Flow B enriches context and evidence. It does not replace structured truth.

---

7. Entity 360 read model

Responsibility	File / Area
Read-time fusion	`services/workflow/src/views.ts`

Workspace transforms	`services/workflow/src/transformers/`

What to trace

- Spine truth fetch
- Knowledge/context fetch
- signal fetch
- fusion into a read model for workspace and cognitive use

Rule reminder
Entity 360 is read-only fusion, not a write surface.

---

8. Cognitive loop

Responsibility	File / Area
Proposal generation / reasoning	`services/think/src/`
Policy and approval rules	`services/govern/src/`
Action execution	`services/act/src/`

What to trace

- Think creates grounded proposals
- Govern evaluates policy
- approval token / approval state is enforced
- Act only runs after approval
- outcome is returned for re-ingestion

---

9. Flow C (AI sessions, MCP, governed AI memory)

Responsibility	File / Area
MCP ingress	`services/mcp-connector/src/`
AI reasoning / triage patterns	`services/think/src/`
Governed proposal path	`services/govern/src/`, `services/act/src/`

What to trace

- AI session capture
- proposal creation instead of direct truth write

- user review / governance path
- approved action execution
- result re-enters normal pipeline path

Rule reminder
Flow C never writes directly to Spine truth.

---

10. Approval and action

Responsibility	File / Area
Approval policy enforcement	`services/govern/src/index.ts`, `services/govern/src/policies.ts`
Action execution logic	`services/act/src/index.ts`, `services/act/src/lib/execution.ts`

What to trace

- approval requirement checks
- `x-approval-token` validation
- connector execution
- auditability of action path

---

11. Re-ingestion after action

Responsibility	File / Area
Action result queue-back	`services/act/src/index.ts`
Pipeline re-entry	pipeline queue consumers

What to trace

- approved action executes externally
- result is not trusted as final truth immediately
- result is queued back into pipeline
- Spine is refreshed through the same canonical path

Rule reminder
Truth must be refreshed through re-ingestion, not assumed from outbound execution alone.

---

12. Quick trace path for developers

Use this sequence when debugging any end-to-end issue:

1. `apps/web/...` — where the user initiated the action

2. `services/gateway/...` — how the request was routed
3. `services/connector/...` or `services/loader/...` — how source data entered
4. `services/normalizer/...` — how it was normalized
5. `services/spine-v2/...` — where truth was written
6. `services/workflow/...` — how it appeared in workspace / Entity 360
7. `services/think/...` — how intelligence was generated
8. `services/govern/...` — how approval policy was checked
9. `services/act/...` — how action executed
10. queue / pipeline path again — how result re-entered truth

---

13. One-line developer rule

If you cannot point to the ingest path, the pipeline path, the Spine write path, and the re-ingestion path, the feature is not fully wired.

If you want, I can also merge this directly into the KT note and produce a final **KT v1.0** document in one piece.

The **12 departments** part is correct, but the **entity counts in your table do not match the live `DOMAIN_SPINE_CONFIG` arrays exactly**. The schema file confirms the 12 active department configs, but several of the counts shown here are lower than what is currently defined in `packages/types/src/schema.ts`.

What is safe to state canonically

Use this wording instead:

12 Departments Active

Total: 12 primary runtime department configs are defined in `packages/types/src/schema.ts` under `DOMAIN_SPINE_CONFIG`. Each department defines its own:

- **entity types** — what records are allowed in the Spine path
- **priority fields** — what fields the pipeline prioritizes during extraction
- **organic capabilities** — what the BFF/UI exposes instead of raw entity types
- **workspace emphasis** — what modules and views are emphasized in L1
- **cognitive relevance** — what signals and reasoning patterns become meaningful in L2

Active department configs

1. **CUSTOMER_SUCCESS**
2. **REVOPS**
3. **SALES**
4. **MARKETING**
5. **PRODUCT_ENGINEERING**
6. **FINANCE**
7. **SERVICE**
8. **PROCUREMENT**
9. **IT_ADMIN**

10. **STUDENT_TEACHER**
11. **BIZOPS**
12. **PERSONAL**

Important runtime note

Department selection is not cosmetic. It determines:

- which `entity_types` are allowed by schema
- which fields are prioritized in extraction
- which organic capabilities appear in the UI
- which workspace modules are emphasized
- which signals and reasoning paths become active downstream

Also add this note

Some tenant-facing department labels resolve into shared base configs through `DEPARTMENT_TO_CONFIG_KEY`, for example:

- `SUPPORT → SERVICE`
- `ENGINEERING → PRODUCT_ENGINEERING`
- `PRODUCT → PRODUCT_ENGINEERING`
- `LEGAL → BIZOPS`
- `HR → BIZOPS`
- `SUPPLY_CHAIN → PROCUREMENT`
- `SERVICE_OPS → SERVICE`

Best KT-safe replacement for your last paragraph

> Each department has its own schema-driven runtime behavior: allowed entity types, priority fields, organic capability mapping, workspace emphasis, and downstream cognitive relevance. These are defined in `packages/types/src/schema.ts` through `DOMAIN_SPINE_CONFIG`, `DEPARTMENT_TO_CONFIG_KEY`, and `ORGANIC_CAPABILITY_MAP`.

IntegrateWise — Data Flow KT v1.2

Status: Canonical internal reference
Classification: Internal
Product Statement: IntegrateWise is a Knowledge Workspace over the Spine, empowered by AI and governed by approvals.

Document Control

Field	Value
Document	IntegrateWise Data Flow KT
Version	v1.2
Date	2026-03-23
Owner	IntegrateWise LLP
Audience	Product, Engineering, AI, Connector, Workflow, GTM
Scope	End-to-end data flow from landing and onboarding through Spine, Workspace, Cognitive, approval, action, and re-ingestion
Canonical runtime path	Landing → Auth → L0 Onboarding → L1 Workspace → L2 Cognitive → L3 Platform
Canonical engine loop	Load → Normalize → Store → Think → Review & Approve → Act → Repeat

---

1. Purpose

This KT defines how data should flow through IntegrateWise so every team uses the same mental model for truth, context, intelligence, approval, and execution.

The platform rule is simple:

All durable business value in IntegrateWise must be created through a Spine-based flow.

That means every meaningful path must follow this sequence:

Ingest → Normalize → Store in Spine → Render in Workspace / Entity 360 → Think → Govern / HITL → Act → Re-ingest

---

2. Canonical Principles

2.1 Spine is the only truth layer

The Spine is the canonical, tenant-partitioned source of structured truth. It holds entity identity, operational state, relationships, schema observations, and the durable data used by the workspace and cognitive layers.

2.2 No data bypasses Layer 3

All external data must enter through the controlled backend path before it can appear in the workspace or cognitive layer. There are no direct UI-side truth paths and no side databases for business truth.

2.3 Flow C never writes directly to Spine truth

Flow A and Flow B can create truth through the pipeline. Flow C contributes governed knowledge and decision context. Truth changes from Flow C happen only after approved action and re-ingestion.

2.4 Workspace is not a truth layer

L1 is the operating surface, not the ingestion surface and not the source of truth. It renders Spine-backed runtime projections.

2.5 No action executes without approval

Governance is a hard gate. If policy requires approval, no execution occurs without the proper approval path.

---

3. Runtime Architecture

The official runtime order is:

Experience → Gateway → Workspace Runtime → External Connectivity → Data Plane → SSOT / Spine → Cognitive → AI Providers

Layer summary

- **L0 Onboarding** — Tenant identity, business context, schema direction, connector relevance
- **L1 Workspace** — Where users work: dashboard, modules, approvals, entity views, and signals
- **L2 Cognitive** — Context, IQ Hub, Evidence, Signals, Think, Act, Approval, Governance, Adjust, Audit, Agents, Twin
- **L3 Platform** — Gateway, connectors, loader, pipeline, Spine, knowledge, workflow, and background processing

---

4. End-to-End Flow

```
flowchart TB
  A[Landing] --> B[Auth]
  B --> C[L0 Onboarding]
  C --> D[Connector Selection and Auth]
  D --> E[Creamy Hydration]
  E --> F[Loader and Queue]
  F --> G[8-Stage Pipeline]
  G --> H[Spine Write]
  H --> I[L1 Workspace Ready]
  H --> J[Entity 360]
  J --> K[L2 Cognitive]
  K --> L[Think]
  L --> M[Govern / HITL]
  M --> N[Act]
  N --> O[Re-ingest]
  O --> G
```

Expanded meaning

1. User lands and authenticates
2. Onboarding seeds tenant configuration and schema direction

3. Connector setup establishes real external bindings
4. Creamy hydration fetches the first useful data
5. Loader enqueues records into the data plane
6. The 8-stage pipeline normalizes every record
7. Spine stores canonical truth
8. Workspace becomes usable from Spine-backed data
9. Entity 360 fuses truth, context, and signals
10. Cognitive services propose actions
11. Govern and HITL gate execution
12. Approved actions execute externally
13. Results return through re-ingestion so truth stays current

---

5. Onboarding and Tenant Setup (L0)

Onboarding determines:

- who the tenant is
- industry and department context
- desired workspace mode and goals
- connector relevance
- schema direction
- whether approved knowledge can be linked into Entity 360

L0 output

Onboarding seeds tenant runtime configuration, especially `tenant_spine_config`, which later drives:

- workspace navigation
- readiness
- allowed entity types
- depth matrix behavior
- module visibility
- schema routing

Core rule

L0 introduces reality. What is declared during onboarding determines what the system is allowed to ingest, normalize, and render later.

---

6. Connector Setup and Activation

The connector stage should work like this:

1. Connector list is filtered by industry and department
2. User authorizes a provider
3. Installation is bound to tenant

4. Connector state is persisted
5. Initial sync starts with `phase: creamy`

Important point

Connector choice affects scope and schema eligibility, but it does **not** change the core platform path.

All connectors still follow:

Connect → Authenticate → Extract → Queue → Normalize → Spine → Workspace / Entity 360

---

7. The Three Data Flows

7.1 Flow A — Structured Truth Ingestion

Flow A handles structured data from systems like CRM, billing, support, project tools, and operational SaaS apps.

```
flowchart LR
  A[Structured Connector or Webhook] --> B[Loader]
  B --> C[PIPELINE_QUEUE]
  C --> D[8-Stage Pipeline]
  D --> E[Normalizer]
  E --> F[Spine Truth]
  F --> G[Entity 360]
  F --> H[L1 Workspace Views]
```

Role of Flow A

- create canonical business truth
- populate domain tables
- record schema observations
- power lists, dashboards, views, and signals
- support write-back reconciliation after approved actions

Sync phases

- **Creamy** — first useful value, fast
- **Needed** — next meaningful coverage
- **Delta** — continuous incremental freshness

No default full-sync-everything behavior should exist.

7.2 Flow B — Unstructured Context Integration

Flow B handles documents, emails, PDFs, chats, transcripts, and related artifacts.

```
flowchart LR
  A[Docs, Mail, PDF, Chat] --> B[Ingest]
  B --> C[Loader and Pipeline]
  C --> D[Entity Linking]
```

```
    D --> E[Spine References]
    D --> F[Knowledge Chunks and Embeddings]
    E --> G[Entity 360]
    F --> G
```

Role of Flow B

- enrich context without replacing structured truth
- link artifacts to Spine entities
- create evidence-backed views
- support retrieval, IQ Hub, and grounded reasoning

Rule

Flow B may enrich truth with references and structured metadata, but it must not overwrite canonical structured business fields with inferred content.

7.3 Flow C — AI Sessions, MCP, and Approved Memory

Flow C handles AI sessions, MCP sessions, external AI content, and human-AI memory.

```
flowchart LR
  A[MCP, AI Chat, Custom AI] --> B[D1 Buffer or Capture Layer]
  B --> C[Triage]
  C --> D{User Decision}
  D -->|Approve| E[Compounding Space]
  D -->|Discard| F[Archive]
  D -->|Defer| C
  E -. optional .-> G[Entity 360]
  C -. action path .-> H[Think -> Govern -> HITL -> Act]
  H --> I[Pipeline Re-ingest]
  I --> J[Spine]
```

Rule

Flow C never writes directly to Spine truth.

It contributes knowledge, context, and decision support. If it triggers a business action, that action must go through approval, execution, and then re-enter the standard data plane before truth updates.

---

8. The Mandatory 8-Stage Pipeline

Every meaningful record must pass through the same canonical data plane:

1. Analyze
2. Classify
3. Filter
4. Refine
5. Extract

6. Validate
7. Sanity
8. Sectorize

Why this matters

This is the platform's normalization contract. It ensures:

- no raw source payload becomes truth directly
- tenant and schema rules are applied consistently
- only canonical shapes reach the Spine
- Flow A and Flow B remain compatible with Entity 360 and Cognitive

---

9. Spine and Storage Responsibilities

Spine

Stores canonical structured truth:

- entities
- relationships
- schema observations
- operational truth
- approval and action records where needed
- linked references

Knowledge

Stores retrieval-optimized context:

- chunks
- embeddings
- linked unstructured artifacts
- evidence content

Compounding Space / Approved Memory

Stores approved Flow C knowledge and memory. This is not SSOT. It is optional contextual augmentation.

Audit / Decisions / Actions

Stores:

- approvals
- denials
- action history
- traceability
- decision memory

Important rule

These stores are not interchangeable. The workspace must not treat Knowledge or approved memory as the primary truth source.

---

10. Workspace Layer Data Loading (L1)

This section clarifies the missing piece that causes the most confusion.

The Workspace Layer does not load from connectors directly.

L1 becomes usable only after Spine-backed data exists.

10.1 The workspace loading principle

L1 is:

- the operating surface
- the place where users work
- the place where signals, approvals, and context are surfaced

L1 is **not**:

- the ingestion layer
- the truth layer
- a direct connector-read layer

It renders data from:

- Spine truth
- workflow / BFF projections
- Knowledge links
- Signals
- optional approved memory when explicitly linked into Entity 360

10.2 What makes L1 ready

The workspace becomes usable after **Creamy** has successfully passed through:

connector auth → loader → queue → 8-stage pipeline → normalizer → Spine write

Only then should the workspace hydrate from canonical data.

10.3 The exact L1 loading chain

```
flowchart LR
  A[Auth and Onboarding] --> B[Connector Auth]
  B --> C[Creamy Sync]
  C --> D[Loader]
  D --> E[Queue]
  E --> F[Pipeline]
  F --> G[Spine Write]
  G --> H[BFF / Readiness / Tenant Config]
  H --> I[WorkspaceShell]
```

```
    I --> J[Dashboard and Modules]
    G --> K[Entity 360]
    K --> L[Signals and L2 Activation]
```

10.4 The four loading surfaces inside L1

A. Workspace shell load

Includes:

- shell
- sidebar / nav
- tenant context
- active view
- personal vs work mode
- readiness state

B. Home dashboard load

Includes:

- dashboard cards
- approvals count
- intelligence count
- focus items
- summary widgets

C. Module data load

Includes workspace module views such as:

- Home
- Accounts / Projects
- Contacts
- Meetings
- Docs
- Tasks
- Calendar
- Notes
- Knowledge Space
- Team
- Pipeline
- Risks
- Expansion
- Intelligence-linked module states

D. Intelligence overlays back into L1

Includes:

- banners
- signal indicators
- attention queues
- approvals
- feed summaries

10.5 Official L1 data sources

The workspace reads from:

1. **Spine**
2. **Knowledge**
3. **Approved memory / compounding space** when explicitly linked
4. **Signals** generated from normalized truth and Think

10.6 Phased workspace loading

Phase 1 — Shell ready

Auth, tenant, nav, and readiness resolve.

Phase 2 — Creamy workspace ready

First useful data lands, and the user can work.

Phase 3 — Module hydration

Dashboard and modules fill from runtime projections.

Phase 4 — Needed expansion

Historical and broader entity coverage expands.

Phase 5 — Delta freshness

Incremental sync keeps L1 current.

10.7 Immediate vs later loads

Immediately after Creamy

- shell and nav
- dashboard cards
- first entity projections
- first approvals and signal feed

Later through Needed / Delta

- deeper history
- broader entity coverage
- richer module views
- stronger Entity 360
- better cognitive quality

10.8 L1 and L2 dependency

- **L1 can render after Creamy**
- **L2 should only fully activate after Creamy and enough meaningful connectors are loaded**

10.9 User-visible interpretation

1. Onboarding decides the lens
2. Connector auth makes the data real
3. Creamy gives first value fast
4. Spine becomes the source
5. Workspace renders that truth
6. Needed and Delta deepen the workspace silently
7. L2 activates when enough truth exists

---

11. Entity 360

Entity 360 is the read-time fusion layer for the platform.

It combines:

- Spine truth
- Knowledge context
- Signals
- optional approved memory when linked

Rule

Entity 360 is a **read model**, not a write model.

It exists to support:

- workspace understanding
- grounded reasoning
- evidence-backed proposals
- consistent entity views

---

12. Cognitive Layer (L2)

The cognitive loop is:

Entity 360 → Think → Govern → HITL → Act → Adjust

```
flowchart LR
  A[Entity 360] --> B[Think]
  B --> C[Govern]
  C --> D[HITL]
  D --> E[Act]
  D --> F[Adjust]
  E --> G[Re-ingest]
  G --> H[Spine]
```

```
    F --> B
```

Think

Reads canonical truth and context to generate signals, proposals, and evidence.

Govern

Evaluates policy and determines whether the proposal can proceed.

HITL

Captures explicit human approval, rejection, or deferment.

Act

Executes approved actions via controlled integration paths.

Adjust

Learns from rejections, outcomes, and decision memory.

---

13. Approval and Re-ingestion Model

No action is complete until its result has been reconciled through the platform.

Correct action path

1. Proposal created
2. Govern checks policy
3. User approves in UI
4. Act executes external mutation
5. Result re-enters the pipeline
6. Spine updates
7. Workspace refreshes from canonical truth

This prevents internal truth from drifting away from real external state.

---

14. Anti-Patterns to Avoid

1. Direct source-to-truth writes
2. AI directly mutating Spine truth
3. Shadow schemas in modules or connectors
4. Workspace reading raw connector APIs directly
5. Entity 360 becoming a hidden write surface
6. Approval bypass for governed actions

---

15. Acceptance Checklist

Tenant and onboarding

- tenant config is seeded correctly
- onboarding choices define scope and schema direction
- workspace shell uses tenant config and readiness

Connector and sync

- connector auth binds correctly to tenant
- connection success triggers Creamy
- source data enters the approved queue path

Pipeline and truth

- all records pass through all 8 stages
- truth writes land in Spine
- Flow B writes linked Knowledge context
- Flow C remains knowledge-first until approved action

Workspace loading

- L1 shell reads BFF / readiness / tenant config
- dashboard uses Spine-backed runtime projections
- connectors do not directly populate L1
- Creamy is the gate for usable workspace
- Needed and Delta deepen the workspace in the background
- signals in L1 come from normalized data and Think
- L2 activation is gated until enough truth exists

Cognitive and action

- Entity 360 is read-time fusion only
- Think reads canonical truth
- Govern and HITL gate action
- Act results re-enter the pipeline
- final workspace state reflects reconciled truth

---

16. One-Sentence KT Summary

IntegrateWise data must always enter through controlled backend ingest paths, pass through the mandatory normalization pipeline, become canonical truth in the Spine, surface into the workspace through Spine-backed runtime projections, power Entity 360 and the cognitive loop, route actions through approval, and return through re-ingestion so the Spine remains the only source of truth.

Appendix A — Visual Architecture Diagrams

A1. L0 → L1 → L2 → L3 View

```
flowchart TB
  A[L0 Onboarding] --> B[L1 Workspace]
  B --> C[L2 Cognitive]
  C --> D[L3 Platform]
  D --> B
```

A2. Three Flow Model

```
flowchart LR
  A[Flow A Structured] --> S[Spine]
  B[Flow B Unstructured] --> K[Knowledge]
  B --> S
  C[Flow C AI and MCP] --> T[Triage and Approved Memory]
  T -. approved action only .-> S
  S --> E[Entity 360]
  K --> E
  T -. optional link .-> E
```

A3. Workspace Loading View

```
flowchart LR
  A[Connector Auth] --> B[Creamy]
  B --> C[Queue]
  C --> D[Pipeline]
  D --> E[Spine]
  E --> F[BFF Projections]
  F --> G[L1 Workspace]
  E --> H[Entity 360]
  H --> I[L2 Cognitive]
```

Appendix B — Screenshot Mapping Appendix

Note: This appendix maps the screenshots you shared to the runtime model and user-facing explanation. Insert the screenshots into these sections when publishing this KT as a final internal asset.

Screenshot 1 — L1 Workspace Home / Dashboard

What it shows

- Workspace shell
- left navigation
- dashboard cards
- today’s focus
- intelligence feed
- approvals tile
- top alert banner

What it proves

- L1 is the primary operating surface
- intelligence can surface back into L1 before the user opens L2
- dashboard content is projection-based and not raw connector output

Suggested caption

> L1 Workspace Home: the operating surface where users work, review focus items, monitor intelligence summaries, and enter approval-driven workflows.

Screenshot 2 — L2 Cognitive Drawer / Intelligence Engine

What it shows

- bottom-up cognitive drawer
- Context, IQ Hub, Evidence, Signals, Think Engine, Act, Approval, Governance, Adjust, Loop, Audit, Agents, Twin tabs
- Active Spine Nodes panel
- Normalization Health panel

What it proves

- L2 is not a separate product surface; it rises from L1
- cognitive services are evidence-backed and Spine-linked
- normalization health and active node visibility connect L2 back to L3 truth creation

Suggested caption

> L2 Cognitive Drawer: the intelligence layer that reads Spine-backed Entity 360, exposes reasoning and governance, and supports approval-driven action.

Screenshot 3 — L1 Workspace with intelligence feed emphasis

What it shows

- workspace shell with visible attention queues
- intelligence feed integrated into work view
- approvals and insights visible without leaving L1

What it proves

- intelligence is allowed to enhance the workspace without forcing a context switch
- users can stay in L1 and still consume meaningful signals

Suggested caption

> Intelligence in L1: signals and approval-relevant context appear directly in the operating workspace before the user opens L2.

Screenshot 4 — L1 Navigation / Work Layer emphasis

What it shows

- expanded workspace navigation
- operational modules like accounts, contacts, meetings, docs, tasks, calendar, notes, knowledge space, team, pipeline, risks, expansion

What it proves

- L1 is the preserved work layer
- modules are dynamically surfaced from tenant context and Spine-backed runtime projections
- this is the place where daily work happens

Suggested caption

> Preserved Work Layer: L1 is where users execute their daily responsibilities through Spine-driven modules and projections.

Screenshot 5 — L2 tab model and normalization observability

What it shows

- full intelligence tab set
- active node visibility
- normalization health card

What it proves

- L2 is tightly coupled to canonical truth quality
- the cognitive layer depends on normalized Spine-backed data, not raw external payloads

Suggested caption

> Cognitive observability: L2 reflects the health of the normalization and Spine pipeline that powers all reasoning and action proposals.

Appendix C — Publish Notes

Appendix C — Publish Notes

When converting this KT into a final internal handoff artifact:

1. Keep the document title, version block, and canonical product statement on page 1
2. Preserve the three mandatory diagrams:
   - end-to-end flow
   - workspace loading flow
   - cognitive loop
3. Insert the five screenshots in Appendix B with captions
4. Keep the acceptance checklist intact
5. Do not reintroduce product/runtime naming collisions
6. Preserve the rule that the Workspace never loads raw connector data directly

---

Appendix D — Department Activation Matrix

Purpose: Show how onboarding department choice activates schema, extraction priority, UI capability surfaces, and downstream reasoning scope.

D1. Canonical rule

Department selection is not cosmetic. It is a schema-routing decision that determines:

- which entity types are allowed in the Spine path
- which fields are prioritized during extraction
- which organic capabilities are exposed through the BFF/UI
- which workspace modules are emphasized in L1
- which signals and reasoning paths become meaningful in L2

The source of truth for this layer is `packages/types/src/schema.ts`, especially:

- `DOMAIN_SPINE_CONFIG`
- `DEPARTMENT_TO_CONFIG_KEY`
- `ORGANIC_CAPABILITY_MAP`

D2. Department activation model

Tenant-facing Department / Runtime Config	Base Config Key	Activation Behavior	Typical Workspace Emphasis	Typical Cognitive Emphasis
CUSTOMER_SUCCESS	`CUSTOMER_SUCCESS`	Customer and post-sale truth model with health, renewals, initiatives, risks, stakeholder outcomes	Accounts, engagement, risks, success plans, initiatives, strategy	Health shifts, renewal risk, success actions, stakeholder and initiative reasoning
REVOPS	`REVOPS`	Revenue operating system with pipeline, forecast, quotas, attribution, and planning	Forecasting, pipeline, attribution, planning, revenue metrics	Forecast variance, stage bottlenecks, revenue signal analysis
SALES	`SALES`	Pipeline and seller execution model centered on opportunities, leads, outreach, quotes, and territory	Pipeline, leads, activities, quotes, territory, competitive intel	Deal momentum, next-best action, outreach effectiveness
MARKETING	`MARKETING`	Campaign, lead, content, audience, and attribution model	Campaigns, lead generation, content, analytics	Channel performance, attribution shifts, campaign optimization

PRODUCT_ENGINEERING	`PRODUCT_ENGINEERING`	Product delivery and engineering operations model with sprints, releases, incidents, repos, and feedback	Delivery, quality, engineering operations, feedback	Release risk, incident reasoning, roadmap and delivery insight
FINANCE	`FINANCE`	Revenue, invoices, payments, expense, budget, reporting, and compliance model	Revenue ops, expenses, budgets, reporting, vendors	Variance, cash / revenue forecasting, compliance and spend signals
SERVICE	`SERVICE`	Support and service operations model with tickets, SLAs, queues, CSAT, and escalations	Tickets, knowledge, quality, service ops	Escalation risk, backlog pressure, SLA and CSAT reasoning
PROCUREMENT	`PROCUREMENT`	Vendor, PO, contract, renewal, spend, and savings model	Vendors, purchasing, spend, renewals	Spend optimization, contract and vendor risk, savings actions
IT_ADMIN	`IT_ADMIN`	Systems, devices, incidents, vulnerability, backup, network, and licensing model	Infrastructure, security, IT ops	Vulnerability pressure, uptime risk, certificate / license / change alerts
STUDENT_TEACHER	`STUDENT_TEACHER`	Education model with students, courses, assignments, grades, attendance, and interventions	Students, courses, engagement, analytics	Student risk, attendance patterns, intervention support
BIZOPS	`BIZOPS`	Cross-functional business operations model with workflow, KPI, OKR, project, and initiative visibility	Strategy, projects, reporting, coordination	Cross-functional bottlenecks, KPI drift, resource / initiative reasoning
PERSONAL	`PERSONAL`	Individual productivity and reflection model	Tasks, goals, calendar, reflection	Focus, planning, execution support, daily decision quality

D3. Department alias / resolution layer

Some tenant-facing department IDs resolve into shared base config families through `DEPARTMENT_TO_CONFIG_KEY`. This keeps onboarding simpler while preserving a coherent runtime schema family.

Tenant-facing Label	Resolves To Base Config
`SUPPORT`	`SERVICE`
`ENGINEERING`	`PRODUCT_ENGINEERING`

`PRODUCT`	`PRODUCT_ENGINEERING`
`LEGAL`	`BIZOPS`
`HR`	`BIZOPS`
`SUPPLY_CHAIN`	`PROCUREMENT`
`SERVICE_OPS`	`SERVICE`
`CSM`	`CUSTOMER_SUCCESS`
`PROJECTS`	`BIZOPS`

D4. Runtime sequence for department activation

```
flowchart LR
  A[Onboarding Department Choice] --> B[normalizeDepartmentId]
  B --> C[DEPARTMENT_TO_CONFIG_KEY]
  C --> D[DOMAIN_SPINE_CONFIG]
  D --> E[Resolved Schema]
  E --> F[8-Stage Pipeline Rules]
  E --> G[Organic Capability Map]
  G --> H[L1 Workspace Emphasis]
  E --> I[L2 Cognitive Relevance]
```

D5. KT-safe statement

> Department choice defines the base runtime schema family. It controls what the pipeline accepts, what the Spine stores, what the BFF translates into organic workspace capabilities, and what the cognitive layer can reason over.

Appendix E — Industry Override Matrix

Purpose: Show how industry selection specializes a department base schema through additive overrides.

E1. Canonical rule

Industries are not standalone workspace families. They are a schema override layer applied on top of the department base config.

The source of truth for this layer is `INDUSTRY_SPINE_BASE` in `packages/types/src/schema.ts`.

Each industry can define:

- `entity_types_extra`
- `priority_fields_override`

The final runtime schema is resolved by `getSpineConfig(industry, department)`.

E2. Industry override model

Industry	Override Behavior	Example Impact on Runtime
SAAS_TECH	No additional override payload in current base	Uses department base schema as-is
PROFESSIONAL_SERVICES	Adds `contract`, `project`, `timesheet`	Strengthens services / delivery / utilization visibility
HEALTHCARE	Adds `contract`, `compliance_item`, `patient_encounter`; overrides account / contract priorities	Extends compliance-sensitive and regulated-operating-state visibility
EDUCATION	Adds `student`, `course`, `enrollment`	Extends academic entities and learning context
MANUFACTURING	Adds `work_order`, `supplier`, `inventory_snapshot`	Extends operational supply and production context
AUTOMOTIVE	Adds `vehicle`, `part`, `warranty`	Extends service / product lifecycle context around vehicles and parts
RETAIL_COMMERCE	Adds `order`, `inventory`, `sku`	Extends commerce and merchandising visibility

FINANCIAL_SERVICES	Adds `compliance_item`, `risk_assessment`; overrides account priorities	Extends regulated finance and risk-aware account context
LOGISTICS	Adds `shipment`, `carrier`, `warehouse`	Extends movement, fulfillment, and storage context
MEDIA	Adds `content_asset`, `license`, `royalty`	Extends publishing / IP / rights management context
PUBLIC_SECTOR	Adds `compliance_item`, `contract`, `grant`; overrides account priorities	Extends procurement, grant, and public compliance visibility

E3. Runtime sequence for industry specialization

```
flowchart LR
  A[Department Base Config] --> B[Industry Selected]
  B --> C[INDUSTRY_SPINE_BASE]
  C --> D[entity_types_extra + priority_fields_override]
  D --> E[getSpineConfig(industry, department)]
  E --> F[Resolved Schema]
  F --> G[Pipeline Filtering and Extraction]
  F --> H[Spine Write Shape]
  F --> I[Workspace and Cognitive Specialization]
```

E4. Practical examples

Combination	Result
CUSTOMER_SUCCESS + SAAS_TECH	CS base config with no additional industry extension
CUSTOMER_SUCCESS + HEALTHCARE	CS base config plus healthcare compliance / encounter-related expansion
FINANCE + FINANCIAL_SERVICES	Finance base config plus risk / compliance-heavy finance specialization

BIZOPS + PROFESSIONAL_SERVICES	BizOps base config plus project / contract / timesheet specialization
SERVICE + RETAIL_COMMERCE	Service base config plus order / inventory / SKU context as relevant linked truth

E5. KT-safe statement

> Departments define the base schema family; industries specialize that family through additive runtime overrides. Industry choice changes what extra entities are recognized, what fields matter most during extraction, and how the same department behaves across different business contexts.

Appendix F — Final Activation Summary

```
flowchart TB
  A[Onboarding] --> B[Department Choice]
  A --> C[Industry Choice]
  B --> D[Department Base Schema]
  C --> E[Industry Override]
  D --> F[Resolved Runtime Schema]
  E --> F
  F --> G[8-Stage Pipeline]
  F --> H[Spine Truth Shape]
  F --> I[Organic Capabilities]
  I --> J[L1 Workspace Emphasis]
  F --> K[L2 Cognitive Relevance]
```

Final rule:

- **Department** chooses the base schema family
- **Industry** modifies it through additive override
- **Resolved schema** controls pipeline acceptance, Spine write shape, workspace emphasis, and cognitive reasoning scope